

Complete Regular Expression (RegEx) Reference Manual

Table of Contents

- 1. [Basic Syntax](#)
 - 2. [Character Classes](#)
 - 3. [Quantifiers](#)
 - 4. [Anchors and Boundaries](#)
 - 5. [Groups and Capturing](#)
 - 6. [Alternation](#)
 - 7. [Lookahead and Lookbehind](#)
 - 8. [Flags and Modifiers](#)
 - 9. [Escape Sequences](#)
 - 10. [Common Patterns](#)
 - 11. [Advanced Techniques](#)
 - 12. [Language-Specific Notes](#)
-

Basic Syntax

Literal Characters

- **Pattern:** `hello`
- **Matches:** The exact string “hello”
- **Example:** `hello world` → matches “hello”

Metacharacters

Special characters with special meanings: `. ^ $ * + ? { } [ ] \ | ( )`

Character Classes

Basic Character Classes

Pattern	Description	Example
<code>.</code>	Any character except newline	<code>h.llo</code> matches “hello”, “hallo”
<code>[abc]</code>	Any character in brackets	<code>[aeiou]</code> matches any vowel
	Any character NOT in	<code>[^aeiou]</code> matches

[^abc]	brackets	consonants
[a-z]	Character range	[a-z] matches lowercase letters
[0-9]	Digit range	[0-9] matches any digit

Predefined Character Classes

Pattern	Description	Equivalent
\d	Digit	[0-9]
\D	Non-digit	[^0-9]
\w	Word character	[a-zA-Z0-9_]
\W	Non-word character	[^a-zA-Z0-9_]
\s	Whitespace	[\t\n\r\f\v]
\S	Non-whitespace	[^ \t\n\r\f\v]

POSIX Character Classes

Pattern	Description
[:alnum:]	Alphanumeric characters
[:alpha:]	Alphabetic characters
[:digit:]	Digits
[:lower:]	Lowercase letters
[:upper:]	Uppercase letters
[:punct:]	Punctuation
[:space:]	Whitespace characters

Quantifiers

Basic Quantifiers

Pattern	Description	Example
*	0 or more	ab* matches “a”, “ab”, “abbb”
+	1 or more	ab+ matches “ab”, “abbb” (not “a”)
?	0 or 1 (optional)	ab? matches “a”, “ab”
{n}	Exactly n times	a{3} matches “aaa”
{n, }	n or more times	a{2, } matches “aa”,

<code>{n,m}</code>	Between n and m times	<code>"aaa", "aaaa"</code> <code>a{2,4}</code> matches <code>"aa", "aaa", "aaaa"</code>
--------------------	-----------------------	---

Greedy vs Lazy Quantifiers

Greedy	Lazy	Description
<code>*</code>	<code>*?</code>	Match as few as possible
<code>+</code>	<code>+</code>	Match as few as possible
<code>?</code>	<code>??</code>	Prefer no match
<code>{n,m}</code>	<code>{n,m}?</code>	Match closer to n than m

Example: In string "hello", pattern `".*"` matches the entire string, while `".*?"` matches empty string.

Anchors and Boundaries

Pattern	Description	Example
<code>^</code>	Start of string/line	<code>^hello</code> matches "hello" at start
<code>\$</code>	End of string/line	<code>world\$</code> matches "world" at end
<code>\b</code>	Word boundary	<code>\bcats\b</code> matches "cat" but not "category"
<code>\B</code>	Non-word boundary	<code>\Bcat\B</code> matches "cat" in "concatenate"
<code>\A</code>	Start of string only	Like <code>^</code> but not affected by multiline
<code>\Z</code>	End of string only	Like <code>\$</code> but not affected by multiline
<code>\z</code>	Absolute end of string	End of string, ignoring final newline

Groups and Capturing

Capturing Groups

Pattern	Description	Example
(abc)	Capturing group	(hello) (world) captures two groups
(?<name>abc)	Named capturing group	(?<greeting>hello)
\1	Back-reference to group 1	(hello) \1 matches “hello hello”
(?:\d+)	Non-capturing group	Groups without storing match

Group Examples

```
# Email pattern with groups
([a-zA-Z0-9._%+-]+)@([a-zA-Z0-9.-]+\.[a-zA-Z]{2,})

# Phone number with named groups
(?<area>\d{3})-(?<exchange>\d{3})-(?<number>\d{4})

# Repeated word detection
\b(\w+)\s+\1\b
```

Alternation

Pattern	Description	Example
	OR operator	cat dog matches “cat” or “dog”
(cat dog)	Grouped alternation	I have a (cat dog)

Examples: - jpe?g|png|gif - matches image file extensions -
(Mr|Mrs|Ms)\.?\s - matches title prefixes

Lookahead and Lookbehind

Lookahead

Pattern	Description	Example
(?=...)	Positive lookahead	hello(=?\sworld) matches “hello” if followed by “ world”
(?!...)	Negative lookahead	hello(?!\sworld) matches “hello” if NOT followed by “ world”

Lookbehind

Pattern	Description	Example
(?<=...)	Positive lookbehind	(?<=hello\s)world matches “world” if preceded by “hello”
(?<![...])	Negative lookbehind	(?<![hello\s)world matches “world” if NOT preceded by “hello”

Advanced Examples

```
# Password validation (8+ chars, 1 uppercase, 1 lowercase, 1 digit)
^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)[a-zA-Z\d@$_!%*?&]{8,}$

# Extract domain from email
(?<=@)[a-zA-Z0-9.-]+

# Match numbers not preceded by letters
(?<![a-zA-Z])\d+
```

Flags and Modifiers

Flag	Name	Description	Example
i	Ignore case	Case-insensitive matching	/hello/i matches “HELLO”
g	Global	Find all matches	/cat/g finds all “cat”
m	Multiline	^ and \$ match line breaks	/^line/m
s	Dot all	. matches newlines	/a.b/s matches “a”
x	Extended	Ignore whitespace, allow comments	/a b/x
u	Unicode	Full Unicode support	/\u{1F600}/u

Escape Sequences

Special Characters to Escape

When you want to match these literally, escape with backslash: . ^ \$ * + ? { } [] \ | ()

Common Escape Sequences

Pattern	Description
\\	Literal backslash
\.	Literal period
*	Literal asterisk
\+	Literal plus
\?	Literal question mark
\	Literal pipe
\(	Literal opening parenthesis
\)	Literal closing parenthesis
\[	Literal opening bracket
\]	Literal closing bracket

Control Characters

Pattern	Description
\t	Tab character
\n	Newline
\r	Carriage return
\f	Form feed
\v	Vertical tab
\0	Null character

Common Patterns

Email Validation

```
# Basic email
^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$

# More comprehensive
^[a-zA-Z0-9.!#$%&'*/+=?^_`{|}~-]+@[a-zA-Z0-9](:[a-zA-Z0-9-]{0,61}[a-zA-Z0-9])?(?:\.[a-zA-Z0-9](:[a-zA-Z0-9-]{0,61}[a-zA-Z0-9])?)*$
```

Phone Numbers

```
# US phone number formats
^((?([0-9]{3})\))?([-. ]?(?([0-9]{3}))[-. ]?(?([0-9]{4}))$

# International format
^\+[1-9]\d{1,14}$
```

URLs

```
# Basic URL
^https?:\/\/[^\s/$. ?#].[^\s]*$
```

```
# More comprehensive
^(https?|ftp):\\\/[^\\s/$. ?#].[^\\s]*$
```

Dates

```
# MM/DD/YYYY or MM-DD-YYYY
^(0[1-9]|1[012])[ - /](0[1-9]|12)[0-9]|3[01])[ - /](19|20)d\d$

# YYYY-MM-DD (ISO format)
^d{4}-(0[1-9]|1[0-2])-(0[1-9]|12)d|3[01])$
```

Credit Card Numbers

```
# Visa
^4[0-9]{12}(?:[0-9]{3})?$

# MasterCard
^5[1-5][0-9]{14}$

# American Express
^3[47][0-9]{13}$
```

IP Addresses

```
# IPv4
^((25[0-5]|2[0-4][0-9]|[01]?[0-9][0-9]?)\\.){3}(25[0-5]|2[0-4][0-9]|
[01]?[0-9][0-9]?)$

# IPv6
^([0-9a-fA-F]{1,4}:){7}[0-9a-fA-F]{1,4}$
```

File Extensions

```
# Image files
\\. (jpg|jpeg|png|gif|bmp|svg)$

# Document files
\\. (pdf|doc|docx|txt|rtf)$
```

Password Validation

```
# At least 8 characters, 1 uppercase, 1 lowercase, 1 digit, 1
special char
^(?=.*[a-z])(?=.*[A-Z])(?=.*\\d)(?=.*[@$!%*?&])[A-Za-z\\d@$!%*?&]{8,}$
```

Advanced Techniques

Recursive Patterns (PCRE)

```
# Balanced parentheses
\\((?:[^()]|(?R))*\\)
```

```
# Nested HTML tags
<([a-zA-Z][a-zA-Z0-9]*)\b[^>]*>(?:[<]|<(?!/?\1>)|(R))*</\1>
```

Conditional Patterns

```
# If group 1 exists, match pattern A, else pattern B
(?:1)pattern_a|pattern_b)
```

```
# Example: optional area code in phone numbers
^(\d)?[0-9]{3}(\d)\d{3}-\d{4}$
```

Atomic Groups

```
# Prevent backtracking
(?:>pattern)
```

```
# Example: efficient matching of quoted strings
"(?>\\.|"[^"]\\")"
```

Language-Specific Notes

JavaScript

```
// Creating regex
const regex1 = /pattern/flags;
const regex2 = new RegExp('pattern', 'flags');

// Common methods
string.match(regex) // Returns matches
string.replace(regex, replacement)
string.search(regex) // Returns index
regex.test(string) // Returns boolean
regex.exec(string) // Returns detailed match info
```

Python

```
import re

# Common functions
re.search(pattern, string) # Find first match
re.findall(pattern, string) # Find all matches
re.sub(pattern, repl, string) # Replace matches
re.split(pattern, string) # Split by pattern

# Compile for efficiency
pattern = re.compile(r'regex_here')
pattern.search(string)
```

PHP

```
// PCRE functions
preg_match($pattern, $string) // Find first match
preg_match_all($pattern, $string) // Find all matches
preg_replace($pattern, $replacement, $string)
```

```
preg_split($pattern, $string)           // Split by pattern
```

Java

```
import java.util.regex.Pattern;
import java.util.regex.Matcher;

Pattern pattern = Pattern.compile("regex_here");
Matcher matcher = pattern.matcher(string);

// Common methods
matcher.find()           // Find next match
matcher.matches()        // Entire string matches
matcher.group()           // Get matched text
```

Testing and Debugging Tips

Online Testing Tools

- [regex101.com](#) - Detailed explanations and testing
- [regexr.com](#) - Visual regex builder
- [regexpal.com](#) - Simple testing interface

Debugging Strategies

1. **Start simple:** Build pattern incrementally
2. **Use groups:** Isolate problematic parts
3. **Test edge cases:** Empty strings, special characters
4. **Watch for greediness:** Use lazy quantifiers when needed
5. **Validate boundaries:** Check start/end anchors

Performance Considerations

- Avoid catastrophic backtracking with nested quantifiers
 - Use atomic groups or possessive quantifiers when appropriate
 - Consider string methods for simple operations
 - Compile regex patterns when using repeatedly
-

Quick Reference Card

Most Used Patterns

<code>\d+</code>	# One or more digits
<code>\w+</code>	# One or more word characters
<code>\s+</code>	# One or more whitespace
<code>^.*\$</code>	# Entire line
<code>\b\w+\b</code>	# Whole words
<code>[a-zA-Z]+</code>	# Letters only
<code>\d{1,3}</code>	# 1 to 3 digits

```
. *?      # Non-greedy any character  
(?i)pattern  # Case insensitive (some flavors)
```

Remember

- RegEx is powerful but can be complex - start simple
- Always test your patterns thoroughly
- Consider readability vs. complexity
- Some problems are better solved without regex
- Different regex engines have slight variations

This manual covers the most common regex features. For advanced or engine-specific features, consult the documentation for your specific programming language or tool.